Design-Divergence Extractor

Tool specification for the data collection of the Beyond Refactorings study

Tool	`dde` (working name)	Version	spec v1.3, 28 Oct 2026
Companion	Workplan v3, Sections 6–8	Target	v0 (packets) Oct 30, v1 Nov 2

This document specifies the standalone script that collects the design-divergence indicators for the study. It is the authority on what each indicator means and how it is computed; the paper's methodology section is written from it.

> **What changed since v1 (16 Oct).** *v1.3:* data preparation moves to a companion tool, `dde-prep` (Section 11); `dde` keeps only extraction, and must never import it. *v1.2:* implementation has not started, so the plan is re-dated and split so that coder packets (stages 1–6) ship before the indicators (stages 7–8); pilot coding no longer waits for I1–I8 (Section 13). *v1.1:* (1) Rename maps are built before file correspondence and matching, in both the pipeline (Section 3) and the implementation plan (Section 13); v1 used them before they existed. (2) The state `measured` is replaced by `divergent`, so every computed value is either `divergent` or `zero`, never both (Section 7). (3) Each indicator has a declared measurement level (method, class, or file), and aggregation runs method → class → file → patch instead of per hunk (Section 8). (4) The shared ancestor is computed in one working repository per variant pair that holds both histories (Section 5).

1 Purpose and scope

The extractor reads repository revisions and patch metadata for each case in the study and computes design-divergence indicators between the source context of a patch and its target counterpart, together with the baseline covariates the analysis needs. It also produces the evidence packets coders use for blind, pass-1 coding.

In scope: Java sources; the indicators in Section 6; refactoring covariates from RefactoringMiner [1, 2]; audit output for validation.

Out of scope: detecting MOs (GACPD's role [3]), integrating patches (RePatch's role [4]), judging semantic correctness, and any general-purpose adaptation evidence of the kind SALP collects.

2 Independence and provenance requirements

ID	Requirement	How it is checked
R1	No dependency on SALP: no SALP imports, SAP inputs, SALP pipeline calls, or SALP schemas. Lives in its own repository.	CI fails on any `salp` import or SAP file pattern.
R2	Uses the same repository revisions as RePatch for every case with an outcome label.	Stage 1 compares the manifest's target SHA with the SHA recorded in RePatch's log; mismatches stop the case.
R3	Never reads integration outputs: RePatch results, conflict logs, or outcome labels.	No such input in the CLI; CI test asserts no code path opens them; outcomes are joined only in the analysis scripts.
R4	Uses only pre-integration states: source before and after the patch, target before integration, and the shared ancestor.	Revision roles are explicit in every output row.
R5	Deterministic and reproducible: same inputs give byte-identical outputs.	Determinism test runs twice and compares; run manifest records all versions and input hashes.
R6	Allowed dependencies: Git, tree-sitter with the Java grammar [5], RefactoringMiner, Python standard library, pandas.	Pinned lock file.

Overlap with SALP. Both tools parse Java with tree-sitter and consume RefactoringMiner output, so some techniques overlap; that is expected for any two tools analysing software structure and evolution. The extractor implements only the indicators this study needs, with definitions fixed in this document. The design-divergence prototype written inside SALP on 14 October is not used; the extractor is implemented separately so the independence claim holds in the code, not just the paper.

3 Pipeline

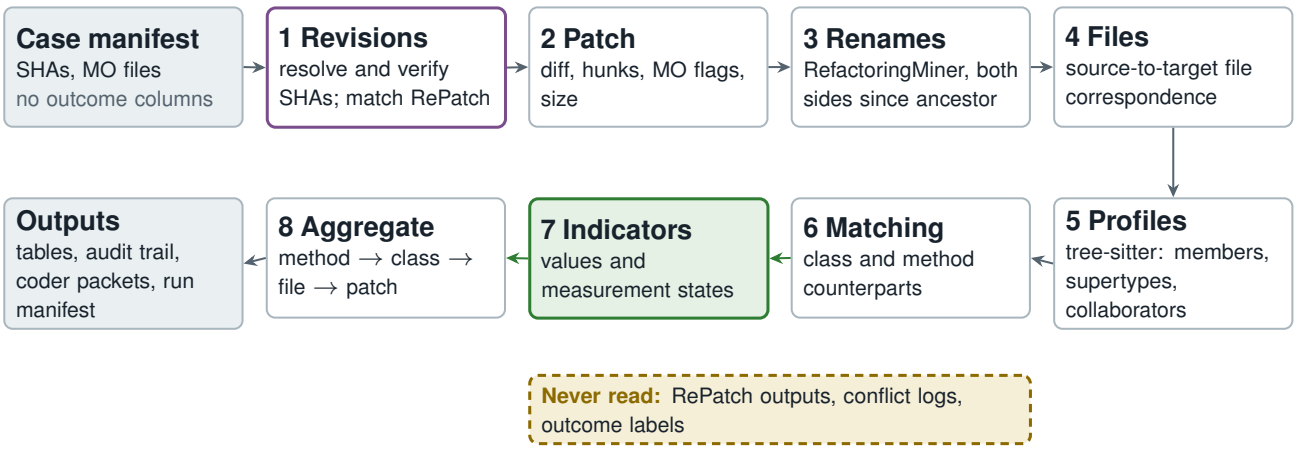


Figure 1: Extractor pipeline, one case at a time. Stage 1 binds the case to RePatch's revisions; stage 3 builds the rename maps that stages 4 and 6 depend on; stage 7 is where indicator values and their measurement states are decided.

1. **Revisions.** Resolve and verify all SHAs (Section 5).
2. **Patch.** Compute the source diff (pre to post), split it into hunks, flag each file as MO or non-MO from the manifest, and record size covariates.
3. **Renames.** Run RefactoringMiner on the ancestor-to-source and ancestor-to-target histories (cached per variant pair) and build rename maps from both sides into the ancestor's names, using only one-to-one rename and

move-and-rename refactorings; a name claimed by two different originals is dropped as ambiguous. This step comes first because file correspondence and matching both depend on the maps.

4. **Files.** Find each MO file's target counterpart: same path; else the path given by a class move or rename in the rename maps; else a unique class of the same name in the target tree; else *unresolved*.
5. **Profiles.** Parse the source-pre, source-post, and target-pre files and build, for each relevant class, its methods (name and arity), fields, supertypes, and collaborators.
6. **Matching.** Match the edited class and method to their target counterparts in ancestor vocabulary (Section 6, I1).
7. **Indicators.** Compute I1–I8 and assign a measurement state to each value.
8. **Aggregate.** Roll method values up to classes, class values up to files, and MO-file values up to the patch (Section 8).

4 Inputs

Case manifest (`cases.csv`), one row per directed transfer attempt, produced in Stage A of the workplan with `dde-prep` (Section 11). It deliberately has *no* outcome columns; `dde` owns its schema and refuses outcome-like columns.

Column	Meaning
`case_id`, `mo_id`	Study case identifier and GACPD MO identifier.
`variant_pair`, `direction`	Pair name and source-to-target direction.
`source_repo`, `target_repo`	Repository URLs or local clone names.
`pr_number`	Source PR, where applicable.
`source_pre_sha`, `source_post_sha`	Source revision before and after the patch.
`target_pre_sha`	Target revision the integration was attempted on.
`repatch_target_sha`	Target SHA as recorded in RePatch's log, for the R2 check (empty if the case has no RePatch run).
`ancestor_sha`	Shared ancestor or divergence commit.
`mo_files`, `non_mo_files`	Semicolon-separated source paths, from GACPD.
`eligibility`, `exclusion_reason`	Result of the Stage A audit.

Repositories: one working repository per variant pair, holding the full history of both variants (the target added as a remote of the source clone, or both fetched into one bare repository). A single repository is needed so that `git merge-base` can see both histories; refs are pinned by SHA, never by branch name. **RefactoringMiner:** a pinned release, invoked as an external process.

5 Revision protocol

- **Source post:** the patch commit (or PR head) recorded by GACPD and RePatch.
- **Source pre:** its first parent; for a multi-commit PR, the PR base recorded in the data. Never recomputed from the current branch state.
- **Target pre:** the SHA RePatch integrated onto. If RePatch's log does not record it, the case is excluded as "target SHA unknown". Reconstructing it from run dates is allowed only by team decision, and every such case is flagged.
- **Ancestor:** `git merge-base` of source pre and target pre, run in the pair's working repository (Section 4).

Where the histories share no commit, the divergence commit recorded by PaReco/GACPD [6] is used and the case is flagged `ancestor_from_metadata`.

Verification flags (written to `case_status.csv`): any SHA missing; target SHA differs from RePatch's (R2); the source diff's files differ from the manifest's; `fix_possibly_present` when the lines added by the MO hunks already appear in the target counterpart file, indicating the fix may have been applied independently since GACPD's classification.

6 Indicators

Notation: for the edited class, M_s and M_t are its method sets in source pre and target pre, as (name, arity) pairs in ancestor vocabulary; F_s, F_t field names; C_s, C_t collaborator types; S_s, S_t supertypes. For the edited method, f_s is its source-pre version and f'_s its source-post version. Each indicator lists the coded phenomenon it is validated against (workplan Section 8).

Measurement level. Each value is computed once per entity at the level where the indicator is defined, not per hunk. *Method level*: I1 `method_match`, I5, I6, computed for every method a MO hunk falls in (a hunk spanning two methods counts for both). *Class level*: I1 `class_match`, I2, I3, I4, I7, computed for the innermost class enclosing each edited method, or enclosing the edit itself when it lies outside any method (a field declaration or initializer block). Nested and secondary top-level classes are separate entities. *File level*: I8.

Indicator (columns)	Definition	Validated against
I1 Counterpart *(method, class)* `class_match`, `method_match`	Class: `same_name`, `renamed`, `unique_elsewhere`, `absent`. Method, in order of preference: `exact` signature, `renamed` (exact after rename map), `name_arity`, `name_only`, `ambiguous` (several candidates), `absent` [7].	Feature absence vs. moved responsibility vs. refactoring
I2 Member overlap *(class)* `method_jaccard`, `field_jaccard`	$J(M_s, M_t)$ and $J(F_s, F_t)$, where $J(A, B) = \|A \cap B\|/\|A \cup B\|$ and $J(\emptyset, \emptyset) = 1$ [8, 9].	Manual role-stereotype change [10, 11]
I3 Relocated methods *(class)* `n_lost`, `n_relocated`	$n_{lost} = \|M_s \setminus M_t\|$; $n_{relocated}$ counts lost methods found (same name and arity) in a different target class, located by `git grep` then parsed.	Responsibility split or merge
I4 Collaborator overlap *(class)* `collab_jaccard`	$J(C_s, C_t)$; C holds type names used in the class body plus capitalized static-call receivers, excluding the class itself and its type parameters; same-package types included [9, 12].	Changed collaboration structure
I5 Contract change *(method)* `params_changed`, `return_changed`	For a method matched at `name_arity` or better, whether the declared parameter types or the return type text differ, after rename normalization [13].	API or contract redesign
I6 Unresolved introduced calls *(method)* `n_introduced`, `unresolved_ratio`	Introduced calls = invoked names in f'_s minus those in f_s. A call is unresolved if neither its name nor its rename-mapped name occurs as `name(` anywhere in the target at its pin. A lower bound.	Missing abstraction, API redesign
I7 Hierarchy change *(class)* `supertypes_changed`	$S_s \neq S_t$ after normalization; generic arguments ignored.	Abstraction replaced
I8 Relocation *(file)* `package_changed`, `module_changed`	Package declaration differs; top-level build module (e.g., Gradle subproject) differs [14].	Cross-layer relocation vs. refactoring

Role stereotypes are not computed by v1. They are coded manually in pass 1; I2 is validated against them. If the classifier behind the earlier role-stereotype dataset [15] can be run on these projects, an optional I9 (`role_source`, `role_target`) may be added in v2.

Baseline covariates (not design indicators): refactoring counts by type touching MO files on each side since the

ancestor; files, hunks, added and deleted lines; number of MO and non-MO files; divergence age in days from ancestor to target pre.

7 Measurement states

Every indicator value carries a state, so that "no divergence" is never confused with "not measured".

State	Meaning
`divergent`	A value was computed and shows some divergence (e.g., Jaccard below 1.0, a changed return type, at least one unresolved call).
`zero`	A value was computed and shows none (e.g., Jaccard 1.0, an `exact` match, no unresolved calls).
`not_applicable`	The indicator does not apply (e.g., I5 for an edit outside any method).
`unresolved`	The correspondence it depends on could not be established (e.g., class absent; ambiguous method).
`unavailable`	Measurement failed (parse error, missing revision, RefactoringMiner timeout).

Exactly one state applies to each value. `divergent` and `zero` are the two computed states; the numeric value is kept with both, so "how much" is never lost. For workplan checkpoint 3, the "unavailable" share counts `unavailable` only. `unresolved` is reported alongside it but not counted as missing, because an unresolved correspondence (for example, no counterpart class) is itself evidence for I1.

8 Aggregation

Values roll up one level at a time, each starting from the level where the indicator is defined (Section 6). **Method to class**, **class to file**, and **file to patch** each report two summaries: the most divergent value (minimum overlap, maximum unresolved ratio, any absent counterpart, any change) and the mean, with the number of contributing entities. A class is counted once however many of its methods were edited, so a large edit to one class does not outweigh a small edit to another. File to patch runs over MO files only; non-MO files are counted but excluded from indicator aggregation, since their divergence does not concern the missed fix. States aggregate conservatively: a patch value is `unavailable` if every contributing value is; otherwise the share of unavailable contributions is reported alongside.

9 Outputs

File	Content
`indicators_method.csv`	One row per edited method in an MO file: case, file, class, method, the hunks mapped to it, method-level values and states.
`indicators_class.csv`	One row per edited class in an MO file: class-level values and states, plus method-level roll-ups.
`indicators_file.csv`, `indicators_patch.csv`	Aggregates with the rules above, plus covariates.
`entity_matches.csv`	Audit trail: every source entity, its target counterpart, how it was matched, and the evidence (paths, line ranges, rename-map entries).
`refactorings.csv`	Normalized RefactoringMiner output per side, with the entries used for rename maps marked.
`case_status.csv`	Verification flags from Section 5 and per-stage errors.
`coder_packets/<case>/`	Blind pass-1 evidence: source method before and after, target counterpart region, class summaries on both sides, refactorings since the ancestor. No indicator values, no outcomes, no conflict output.
`run_manifest.json`	Tool commit, Python, tree-sitter and grammar versions, RefactoringMiner version, manifest hash, per-case SHAs, timestamps.

Coder packets are generated by the same tool so that pass-1 coding sees exactly the revisions the indicators were

computed on, while staying blind to both indicator values and outcomes.

10 Validation support

After pass-1 coding is frozen, `dde validate` joins `indicators_patch.csv` with the coded labels and writes `validation_sheet.csv`: for each indicator, the cases where its target code is present or absent, its value, and the entity-match evidence, ready for the agreement counts and error inspection in workplan Section 8. Outcome labels are still not involved.

11 Command line

```
dde verify    --manifest cases.csv --repos repos/           # stage 1 only
dde extract   --manifest cases.csv --repos repos/ --out out/ [--cases C01,C02]
dde packets   --manifest cases.csv --repos repos/ --out packets/
dde validate  --indicators out/indicators_patch.csv --codes codes.csv --out out/
```

Companion: data preparation (dde-prep)

Everything that builds dde's inputs lives in a separate package in the same repository, `dde-prep`, released with `dde`. It may read RePatch results; `dde` may not, and a test fails if any `dde` module imports `dde-prep` (R3). The manifest is the only artifact that crosses from one to the other.

```
dde-prep draft        --gacpd-run GACPD/ --out cases.csv     # duplicates marked
dde-prep setup-repos  --cases cases.csv --repos repos/        # the only network step
dde-prep apply-shas   --cases cases.csv --runs restricted/outcomes.csv --repos repos/
dde-prep reverify     --cases cases.csv --repos repos/ --out verify/ [--apply]
dde-prep check        --cases cases.csv
dde-prep checkpoint1  --cases cases.csv --runs restricted/outcomes.csv --status verify/case_status.csv
dde-prep pilot-sheets | pilot-agreement                      # coding pilot support
```

12 Testing

- **Unit tests per indicator** on small synthetic Java classes, including a rename that must not count as divergence, a moved method that must count as relocated, a hunk outside any method (method-level indicators `not_applicable`), and a file with two edited classes (two class rows).
- **Golden cases:** three hand-verified real cases (at least one L2-like and one L3-like) whose expected values are written down before the tool is run on them.
- **Leakage test:** asserts no code path opens RePatch outputs or outcome files (R3).
- **Independence test:** fails on any `salp` import (R1).
- **Boundary test:** fails if any `dde` module imports `dde-prep` (R3).
- **Determinism test:** two runs, byte-identical outputs (R5).

13 Implementation plan

Packets come first because pass-1 coding is on the critical path and needs only stages 1–6; indicators are not shown to coders, so they can follow while coding runs.

When	Deliverable
Mon Oct 26	Repository skeleton, manifest reader, per-pair working repositories, revision resolution and verification (`dde verify`).
Tue Oct 27	Patch extraction; RefactoringMiner integration and two-sided rename maps. Start the per-pair RefactoringMiner caches today, Kafka first, since they are the slowest step and can run unattended.
Wed Oct 28	File correspondence and tree-sitter profiles. Golden-case expected values written down before any run.
Thu Oct 29	Matching (I1 correspondence) and `dde packets`; unit tests for stages 1–6.
Fri Oct 30	**v0**: coder packets for the pilot sample fixed at workplan checkpoint 1; run manifest. Pass-1 coding starts from these packets.
Oct 31–Nov 1	I1–I8 with states; aggregation; covariates; unit, leakage, independence, and determinism tests.
Mon Nov 2	**v1**: indicators on the pilot set; golden cases checked; fixes from coder feedback on packets.
Tue Nov 3	`dde validate` on frozen pass-1 codes, for workplan checkpoint 3.
Nov 4–7	Runs on all eligible cases; fixes only, no new indicators.

14 Known limitations

- Name-level analysis: tree-sitter resolves no types, so overloads and same-named classes can be confused; ambiguity is reported, not hidden.
- The rename map uses only one-to-one renames; splits and merges remain as divergence, which is intended but can include composite refactorings.
- I6 counts a name as resolved if it occurs anywhere in the target, so it understates unresolved calls.
- Collaborators come from type names in source text, not resolved dependencies.
- RefactoringMiner can time out on long histories (notably Kafka); affected values become `unavailable` and are counted.
- Java only.

References

[1] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 483–494, 2018.

[2] Nikolaos Tsantalis, Ameya Ketkar, and Danny Dig. RefactoringMiner 2.0. *IEEE Transactions on Software Engineering*, 48 (3):930–950, 2022.

[3] GACPD authors (to be confirmed). GACPD: an extension of PaReco for detecting missed patch opportunities (title to be confirmed), 2026. Manuscript in preparation for IEEE TSE; to be posted on arXiv before the Designing 2027 submission.

[4] Daniel Ogenrwot and John Businge. Refactoring-aware patch integration across structurally divergent Java forks. In *Proceedings of the IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2025. doi: 10.1109/SCAM67354.2025.00010.

[5] Tree-sitter contributors. Tree-sitter: An incremental parsing system for programming tools, 2026. URL `https://tree-sitter.github.io/tree-sitter/`. Software, accessed October 2026; record the grammar versions used.

[6] Poedjadevie Kadjel Ramkisoen, John Businge, Brent van Bladel, Alexandre Decan, Serge Demeyer, Coen De Roover, and Foutse Khomh. PaReco: Patched clones and missed patches among the divergent variants of a software family. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2022. doi: 10.1145/3540250.3549112.

[7] Michael W. Godfrey and Lijie Zou. Using origin analysis to detect merging and splitting of source code entities. *IEEE Transactions on Software Engineering*, 31(2):166–181, 2005. doi: 10.1109/TSE.2005.28.

[8] Paul Jaccard. The distribution of the flora in the alpine zone. *New Phytologist*, 11(2):37–50, 1912.

[9] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.

[10] Rebecca Wirfs-Brock and Alan McKean. *Object Design: Roles, Responsibilities, and Collaborations*. Addison-Wesley, 2003.

[11] Daniel Ogenrwot, Joyce Nakatumba-Nabende, John Businge, and Michel R. V. Chaudron. Empirical investigation of the relationship between design smells and role stereotypes. arXiv:2406.19254, 2024.

[12] Lu Xiao, Yuanfang Cai, and Rick Kazman. Design rule spaces: A new form of architecture insight. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pages 967–977, 2014. doi: 10.1145/2568225.2568241.

[13] Danny Dig and Ralph Johnson. How do APIs evolve? A story of refactoring. *Journal of Software Maintenance and Evolution: Research and Practice*, 18(2):83–107, 2006. doi: 10.1002/smr.328.

[14] Duc Minh Le, Pooyan Behnamghader, Joshua Garcia, Daniel Link, Arman Shahbazian, and Nenad Medvidovic. An empirical study of architectural change in open-source software systems. In *Proceedings of the 12th Working Conference on Mining Software Repositories (MSR)*. IEEE, 2015.

[15] Daniel Ogenrwot, Joyce Nakatumba-Nabende, and Michel R. V. Chaudron. Integration of design smells and role-stereotypes classification dataset. *Data in Brief*, 36:107125, 2021. doi: 10.1016/j.dib.2021.107125.